

C++ Módulo 04

Polimorfismo, Gestão de Memória e Classes Abstratas

Leonardo La Rocque

Escola 42 · Documentação Bibliográfica

Introdução

Este documento constitui a referência bibliográfica oficial do teu percurso no **Módulo 04 de C++**. O objetivo deste material é consolidar a teoria por trás da Programação Orientada a Objetos (OOP) avançada em C++98, documentando rigorosamente as soluções arquiteturais desenvolvidas ao longo dos exercícios práticos.

1. Polimorfismo e Dynamic Dispatch

O Polimorfismo (do grego "*muitas formas*") é um dos pilares da OOP. Permite tratar objetos de classes derivadas (como `Dog` ou `Cat`) como se fossem objetos da sua classe genérica de base (`Animal`).

Early Binding vs Late Binding

Por defeito, o compilador C++ utiliza **Resolução Estática (Early Binding)**. Durante a compilação, o programa decide qual função chamar olhando *apenas* para o tipo do ponteiro.

O Problema sem Polimorfismo:

Se instanciarmos `Animal* a = new Dog();` e chamarmos `a->makeSound()` sem o uso de funções virtuais, o compilador ignora a natureza real do objeto (Dog) e chama o som genérico da classe Animal, gerando comportamentos incorretos.

Para resolver este impasse, introduz-se a palavra-chave `virtual`. Esta instrui o compilador a utilizar a **Resolução Dinâmica (Late Binding ou Dynamic Dispatch)**. A decisão de qual função executar é adiada para o tempo de execução (runtime), baseando-se no tipo real do objeto alocado na memória RAM.

Estrutura do Objeto (RAM)	Virtual Table (VTable) do Cão
Objeto: Dog Atributo: type = "Dog" _vptr (Ponteiro Oculto) Aponta dinamicamente para a VTable correta	[0] <code>Dog::~Dog()</code> [1] <code>Dog::makeSound()</code> → Imprime "WOOF" [2] <code>Animal::getType()</code> ----- Sempre que chamamos <code>makeSound()</code>, o programa segue o <code>_vptr</code> e encontra a função exata nesta tabela.

Esquema 1: Relação entre o ponteiro escondido (vptr) e a VTable para resolução dinâmica.

A Regra de Ouro do Destrutor Virtual

Sempre que uma classe possui funções virtuais e é desenhada para herança, o seu **destrutor tem de ser obrigatoriamente virtual** (`virtual ~Animal();`). Se não o for, aplicar o comando `delete` a um objeto derivado através de um ponteiro da classe base resultará num vazamento de memória (Memory Leak), pois apenas a parte base do objeto será destruída.

2. Gestão de Memória: Cópia Rasa vs Profunda

A alocação dinâmica de atributos dentro das classes (como a classe `Brain` instanciada dentro de `Dog`) altera radicalmente a forma como os objetos devem ser copiados em C++.

Shallow Copy (Cópia Rasa / Erro)	Deep Copy (Cópia Profunda / Correto)
O comportamento padrão do C++ copia apenas os endereços de memória dos ponteiros. Dog 1 & Dog 2 ↓ [Partilham o mesmo Brain na RAM] Consequência: Double-Free / Segfault. Quando um dos cães é destruído, o cérebro desaparece. O segundo cão fica com um ponteiro "morto".	A Forma Canónica Ortodoxa obriga à criação manual de novas alocações de memória. Dog 1 ↓ Brain A Dog 2 ↓ Brain B Consequência: Independência Total. Os valores do cérebro são copiados, mas os endereços de memória são únicos.

Para garantir a Cópia Profunda, deve-se implementar o **Construtor de Cópia** alocando memória com o comando `new`, e o **Operador de Atribuição** desreferenciando os ponteiros para invocar o operador da classe subjacente:

```
// Implementação do Operador de Atribuição (Deep Copy)
Dog& Dog::operator=(const Dog& other)
{
    if (this != &other) // Proteção contra auto-atribuição
    {
        this->type = other.type;
        // Desreferenciar garante a cópia do conteúdo e não do endereço
        *this->_brain = *other._brain;
    }
    return (*this);
}
```

3. Classes Abstratas

Ocasionalmente, criam-se classes que servem unicamente como "contratos" (templates arquiteturais) para classes derivadas, não possuindo lógica para existir isoladamente. Um "Animal" genérico não deve ser instanciado.

Em C++, proíbe-se a instanciação de uma classe convertendo-a numa **Classe Abstrata**. Isto alcança-se ao definir, no mínimo, uma função como um **Método Virtual Puro**, acrescentando a sintaxe `= 0`.

O Contrato Arquitetural:

Ao estipular `virtual void makeSound() const = 0;`, a classe diz ao compilador: *"Eu exijo este comportamento, mas delego a implementação aos meus descendentes."* Qualquer classe derivada é obrigada a providenciar o código para esta função, caso contrário, será também ela considerada abstrata.